Python funcional

30 novos exercícios

Lista complementar de nível médio a difícil, seguindo as restrições da lista fornecida. Os enunciados não incluem soluções.

10
nível médio

20
nível difícil

88
exemplos de entrada e saída

01 **Funções puras.** Mesmos argumentos devem produzir o mesmo retorno. Não altere entradas nem estado global; não use input, print, arquivos ou outros efeitos colaterais.

02 **Somente recursão.** Não use for, while, compreensões, geradores ou ferramentas de iteração prontas, incluindo map, filter e reduce.

03 **Definição por casos.** Cada ramo da função deve apenas retornar uma expressão. Use if/elif/else, expressões condicionais, lambda e chamadas de auxiliares. Não use atribuições locais, reatribuições ou o operador :=.

04 **Sem funções ou métodos prontos.** Não use imports nem recursos como len, sum, min, max, sorted, range, zip, enumerate, any, all, set, dict, int, str, abs ou métodos como append, pop e sort. Não use os operadores in e not in.

05 **Acesso às sequências.** Use somente head, tail, init e last para obter elementos ou partes de listas e strings. Essas primitivas são fornecidas e só recebem sequências não vazias. Não use índices, slices ou desempacotamento.

06 **Construção de valores.** São permitidos literais, listas novas, operadores aritméticos, comparações e operadores lógicos. Para listas, + só é permitido se um dos operandos tiver no máximo um elemento. Não use repetição de sequências com *.

07 **Auxiliares e primitivas.** Pode reutilizar funções puras que você implementou. head devolve o primeiro elemento; tail, tudo exceto o primeiro; init, tudo exceto o último; last, o último. Para strings, elementos são caracteres.

08 **Convenções.** Entrada significa chamada da função; saída significa valor retornado. Trate apenas os casos de erro pedidos. As entradas de teste podem ser pequenas: não se exige otimização ou cache. As mesmas regras valem para todos os exercícios.

Percurso: 01-10: transformações e funções de ordem superior; 11-20: algoritmos sobre listas, matrizes e strings; 21-30: busca combinatória, otimização, grafos e árvores.

01 Compactação por blocos consecutivos

MÉDIO

`compactar(lista) / expandir(blocos)`

Implemente duas funções inversas. compactar substitui cada bloco máximo de valores consecutivos iguais pelo par [valor, quantidade]. expandir reconstrói a lista original a partir desses pares. Repetições separadas pertencem a blocos diferentes.

Condições. Os valores são inteiros; as quantidades recebidas por expandir são inteiros positivos. Ambas retornam [] para entrada vazia.

```
Entrada  compactar([2, 2, 5, 2, 2, 2])
Saída    [[2, 2], [5, 1], [2, 3]]

Entrada  expandir([[4, 2], [1, 1], [4, 1]])
Saída    [4, 4, 1, 4]

Entrada  compactar([])
Saída    []
```

02 Rotação circular com deslocamento assinado

MÉDIO

`rotacionar(lista, k)`

Retorne a lista após k rotações de uma posição para a esquerda. Se k for negativo, rotacione para a direita. Deslocamentos maiores que o tamanho da lista devem ser reduzidos a um deslocamento equivalente.

Condições. k é inteiro. Preserve repetições e não altere a entrada. A rotação da lista vazia é [].

```
Entrada  rotacionar([1, 2, 3, 4, 5], 7)
Saída    [3, 4, 5, 1, 2]

Entrada  rotacionar([1, 2, 3, 4], -1)
Saída    [4, 1, 2, 3]

Entrada  rotacionar([], 10)
Saída    []
```

03 Janelas deslizantes completas

MÉDIO

`janelas(lista, k)`

Retorne todas as sublistas contíguas de tamanho k, avançando uma posição por vez. As janelas podem se sobrepor e devem aparecer na ordem dos respectivos inícios.

Condições. k é inteiro positivo. Não complete janelas com valores artificiais. Se a lista tiver menos de k elementos, retorne [].

Entrada `janelas([4, 1, 7, 3], 3)`
Saída `[[4, 1, 7], [1, 7, 3]]`

Entrada `janelas([8, 5], 1)`
Saída `[[8], [5]]`

Entrada `janelas([1, 2], 3)`
Saída `[]`

04 Partição estável por predicado

MÉDIO

`particionar(predicado, lista)`

Receba uma função pura que devolve True ou False e uma lista. Retorne [aceitos, rejeitados], colocando em aceitos os elementos que satisfazem o predicado e em rejeitados os demais. Preserve a ordem relativa dentro de cada grupo.

Condições. A lista contém inteiros. Não avalie o predicado sobre a lista inteira; aplique-o aos elementos durante a decomposição recursiva.

Entrada `particionar(lambda x: x % 2 == 0, [-3, 4, 0, 5, 2])`
Saída `[[4, 0, 2], [-3, 5]]`

Entrada `particionar(lambda x: x > 0, [])`
Saída `[[], []]`

05 Histograma na ordem da primeira ocorrência

MÉDIO

`histograma(lista)`

Produza uma lista de pares [valor, frequência_total]. Cada valor deve aparecer uma única vez no resultado, na ordem em que surge pela primeira vez na entrada. Conte todas as ocorrências, inclusive as que não são consecutivas.

Condições. A entrada é uma lista de inteiros. Não use dicionários, conjuntos ou ordenação para mudar a ordem solicitada.

```
Entrada  histograma([3, 1, 3, 2, 1, 3])
Saída    [[3, 3], [1, 2], [2, 1]]

Entrada  histograma([0, -1, 0, -1])
Saída    [[0, 2], [-1, 2]]

Entrada  histograma([])
Saída    []
```

06 Interseção de multiconjuntos

MÉDIO

`intersecao_multi(a, b)`

Retorne os elementos comuns às duas listas respeitando as multiplicidades: um valor deve ocorrer no resultado tantas vezes quanto o menor de seus totais em a e em b. Preserve a ordem das ocorrências aproveitadas de a.

Condições. As listas contêm inteiros. Ao percorrer a, aproveite uma ocorrência sempre que ainda houver uma ocorrência correspondente disponível em b.

```
Entrada  intersecao_multi([2, 1, 2, 3, 2], [2, 2, 4, 1])
Saída    [2, 1, 2]

Entrada  intersecao_multi([5, 5, 5], [5])
Saída    [5]

Entrada  intersecao_multi([], [1, 2])
Saída    []
```

07 Segmentos estritamente crescentes

MÉDIO

`segmentar_crescentes(lista)`

Divida a lista em segmentos contíguos máximos estritamente crescentes. Um segmento termina quando o próximo elemento é menor ou igual ao último elemento do segmento atual. Nenhum valor pode ser descartado.

Condições. A entrada contém inteiros. A concatenação dos segmentos deve reconstruir a entrada. Para [], retorne [].

Entrada `segmentar_crescentes([1, 3, 2, 2, 4, 0])`
Saída `[[1, 3], [2], [2, 4], [0]]`

Entrada `segmentar_crescentes([3, 2, 1])`
Saída `[[3], [2], [1]]`

Entrada `segmentar_crescentes([])`
Saída `[]`

08 Composição de uma sequência de funções

MÉDIO

`compor(funcoes, x)`

Aplique ao valor x todas as funções puras da lista, da esquerda para a direita: a primeira recebe x, a segunda recebe o resultado da primeira, e assim por diante. Retorne somente o resultado final.

Condições. Cada função recebe e devolve um inteiro. A lista vazia representa a identidade: compor([], x) devolve x. Não use eval ou exec.

Entrada `compor([lambda x: x + 2, lambda x: x * x, lambda x: x - 1], 3)`
Saída `24`

Entrada `compor([lambda x: x * 2, lambda x: x + 3], 4)`
Saída `11`

Entrada `compor([], 9)`
Saída `9`

09 Entrelaçamento de várias listas

MÉDIO

`entrelacar(listas)`

Percorra uma lista de listas em rodadas. Em cada rodada, retire conceitualmente o primeiro elemento de cada lista ainda não vazia, respeitando a ordem das listas. Retorne todos os elementos em uma única lista.

Condições. As listas internas contêm inteiros e podem ser vazias ou ter tamanhos diferentes. Pule listas esgotadas e não altere nenhuma lista recebida.

```
Entrada  entrelacar([[1, 2, 3], [10], [], [20, 21]])
Saída    [1, 10, 20, 2, 21, 3]

Entrada  entrelacar([[1], [], [2, 3]])
Saída    [1, 2, 3]

Entrada  entrelacar([[], []])
Saída    []
```

10 Ocorrências contíguas com sobreposição

MÉDIO

`ocorrencias(padrao, lista)`

Retorne os índices iniciais de todas as ocorrências contíguas de padrao em lista, em ordem crescente. Ocorrências sobrepostas também contam. Os índices começam em zero.

Condições. As entradas são listas de inteiros. O padrão vazio ocorre em todas as fronteiras: para uma lista de tamanho n, sua saída é [0, 1, ..., n].

```
Entrada  ocorrencias([1, 1], [1, 1, 1])
Saída    [0, 1]

Entrada  ocorrencias([], [7, 8])
Saída    [0, 1, 2]

Entrada  ocorrencias([2, 3], [2, 2])
Saída    []
```

11 Merge sort estável de registros

DIFÍCIL

`merge_sort(registros)`

Ordene registros [chave, rótulo] por chave crescente usando merge sort: divisão recursiva, ordenação das partes e intercalação. A ordenação deve ser estável: registros com a mesma chave mantêm sua ordem original.

Condições. As chaves são inteiras e os rótulos são strings. Compare apenas as chaves; não use o rótulo como desempate. A saída de [] é [].

```
Entrada  merge_sort([[2, 'a'], [1, 'b'], [2, 'c'], [1, 'd']])
Saída    [[1, 'b'], [1, 'd'], [2, 'a'], [2, 'c']]

Entrada  merge_sort([[0, 'x'], [0, 'y']])
Saída    [[0, 'x'], [0, 'y']]

Entrada  merge_sort([])
Saída    []
```

12 Seleção sem ordenar a lista inteira

DIFÍCIL

`selecionar(lista, k)`

Retorne o elemento que ocuparia o índice k se a lista fosse ordenada em ordem crescente. Implemente seleção por pivô, particionando recursivamente em menores, iguais e maiores; prossiga apenas na região necessária.

Condições. A lista contém inteiros e k é inteiro, com índices a partir de zero. Repetições ocupam posições distintas. Se k não for um índice válido, retorne None.

```
Entrada  selecionar([7, 2, 5, 2, 9], 2)
Saída    5

Entrada  selecionar([7, 2, 5, 2, 9], 1)
Saída    2

Entrada  selecionar([4, 1], -1)
Saída    None
```

13 Convolução de polinômios

DIFÍCIL

`convoluir(a, b)`

Cada lista representa os coeficientes de um polinômio, do termo constante para o de maior grau. Calcule os coeficientes do produto: o termo de índice k é a soma de a[i] * b[j] sobre os pares de índices cuja soma é k.

Condições. Os coeficientes são inteiros. Para entradas não vazias de tamanhos m e n, retorne m + n - 1 coeficientes, preservando zeros finais. Se alguma entrada for [], retorne [].

Entrada `convoluir([1, 2, 3], [4, -1])`
Saída `[4, 7, 10, -3]`

Entrada `convoluir([0, 1], [0, 1])`
Saída `[0, 0, 1]`

Entrada `convoluir([], [1, 2])`
Saída `[]`

14 Rotação de matriz em 90 graus

DIFÍCIL

`girar_matriz(matriz)`

Retorne uma nova matriz obtida ao girar a entrada em 90 graus no sentido horário. Uma matriz com m linhas e n colunas deve gerar n linhas e m colunas. Construa também uma função auxiliar de transposição recursiva.

Condições. A entrada é [] ou uma matriz retangular de inteiros com pelo menos uma linha e uma coluna. Não use acesso por índices nem zip.

Entrada `girar_matriz([[1, 2, 3], [4, 5, 6]])`
Saída `[[4, 1], [5, 2], [6, 3]]`

Entrada `girar_matriz([[1], [2], [3]])`
Saída `[[3, 2, 1]]`

Entrada `girar_matriz([])`
Saída `[]`

15 Multiplicação recursiva de matrizes

DIFÍCIL

`multiplicar_matrizes(a, b)`

Implemente o produto matricial. Cada elemento da saída deve ser o produto escalar de uma linha de a por uma coluna de b. Decomponha recursivamente o trabalho em linhas, colunas e pares de elementos.

Condições. As matrizes são retangulares, não vazias, contêm inteiros e têm dimensões compatíveis. Você pode reaproveitar a transposição da questão 14.

Entrada `multiplicar_matrizes([[1, 2, 3], [4, 5, 6]], [[1, 0], [0, 1], [1, 1]])`
Saída `[[4, 5], [10, 11]]`

Entrada `multiplicar_matrizes([[0, 2]], [[3], [4]])`
Saída `[[8]]`

16 Percurso de uma matriz em espiral

DIFÍCIL

`espiral(matriz)`

Leia a matriz em espiral no sentido horário, começando no canto superior esquerdo. Percorra a borda externa e repita o processo sobre a região restante. Retorne uma lista simples com todos os elementos.

Condições. A entrada é [] ou uma matriz retangular não vazia de inteiros. Não repita elementos quando restar apenas uma linha ou uma coluna.

Entrada `espiral([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])`
Saída `[1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7]`

Entrada `espiral([[1], [2], [3]])`
Saída `[1, 2, 3]`

Entrada `espiral([])`
Saída `[]`

17 Delimitadores balanceados

DIFÍCIL

`balanceados(texto)`

Verifique se os delimitadores (), [] e {} estão corretamente pareados e aninhados. Ignore todos os outros caracteres. A ordem de fechamento deve corresponder à ordem inversa de abertura.

Condições. A entrada é uma string. Use uma representação imutável para as aberturas pendentes. A string vazia é válida.

```
Entrada  balanceados('a*(b+[c-{d}])')
Saída    True

Entrada  balanceados('([)]')
Saída    False

Entrada  balanceados('')
Saída    True
```

18 Calculadora em notação pós-fixa

DIFÍCIL

`avaliar_posfixa(tokens)`

Avalie uma expressão pós-fixa usando uma pilha imutável. Cada token é um inteiro ou um dos símbolos +, -, *, //. Um operador consome os dois valores mais recentes: o penúltimo é o operando esquerdo e o último, o direito.

Condições. Use a divisão inteira // de Python. Retorne None se faltarem operandos, houver divisão por zero ou não restar exatamente um valor ao final.

```
Entrada  avaliar_posfixa([8, 3, '-', 2, '*'])
Saída    10

Entrada  avaliar_posfixa([5, '+'])
Saída    None

Entrada  avaliar_posfixa([5, 0, '//'])
Saída    None
```

19 Correspondência com curingas

DIFÍCIL

`corresponde(padrao, texto)`

Verifique se o padrão corresponde ao texto inteiro. O símbolo ? representa exatamente um caractere; * representa qualquer sequência de zero ou mais caracteres. Os demais caracteres representam a si mesmos.

Condições. As entradas são strings de caracteres ASCII. Não existe escape para ? ou *. Trate explicitamente padrões e textos vazios.

Entrada `corresponde('a*b?', 'axxbz')`
Saída `True`

Entrada `corresponde('a?b', 'ab')`
Saída `False`

Entrada `corresponde('*', '')`
Saída `True`

20 Distância de edição

DIFÍCIL

`distancia(a, b)`

Calcule o menor número de operações necessárias para transformar a string a na string b. As operações permitidas são inserir um caractere, remover um caractere e substituir um caractere por outro, todas com custo 1.

Condições. As strings usam letras minúsculas sem acentos. Não use bibliotecas de distância de edição nem tabelas modificadas em memória.

Entrada `distancia('kitten', 'sitting')`
Saída `3`

Entrada `distancia('', 'abc')`
Saída `3`

Entrada `distancia('casa', 'casa')`
Saída `0`

21 Maior subsequência comum com desempate

DIFÍCIL

`scm(a, b)`

Retorne uma subsequência comum de comprimento máximo das duas strings. Uma subsequência preserva a ordem dos caracteres, mas pode pular posições. Se houver várias respostas máximas, escolha a lexicograficamente menor.

Condições. As entradas usam letras minúsculas sem acentos. Compare strings com < ou > para o desempate. Se não houver caracteres em comum, retorne a string vazia.

Entrada `scm('abcbdab', 'bdcaba')`
Saída `'bcab'`

Entrada `scm('ba', 'ab')`
Saída `'a'`

Entrada `scm('', 'abc')`
Saída `''`

22 Todos os subconjuntos com soma-alvo

DIFÍCIL

`subconjuntos_soma(lista, alvo)`

Retorne todos os subconjuntos cuja soma é alvo. Cada elemento pode ser usado uma única vez. Preserve a ordem original dos elementos em cada subconjunto. Na busca, explore incluir o elemento atual antes de explorar excluí-lo.

Condições. A lista contém inteiros distintos em ordem crescente, podendo incluir negativos e zero. O subconjunto vazio é uma resposta quando alvo é 0. Não encerre a busca ao atingir a soma antes de decidir sobre todos os elementos.

Entrada `subconjuntos_soma([-2, 0, 2, 3], 0)`
Saída `[[-2, 0, 2], [-2, 2], [0], []]`

Entrada `subconjuntos_soma([2, 4], 3)`
Saída `[]`

Entrada `subconjuntos_soma([], 0)`
Saída `[[]]`

23 Permutações únicas em ordem lexicográfica

DIFÍCIL

`permutacoes(lista)`

Retorne todas as permutações distintas da lista em ordem lexicográfica crescente. Valores repetidos não podem gerar respostas duplicadas. Cada permutação deve usar exatamente as ocorrências da entrada.

Condições. A entrada é uma lista de inteiros em qualquer ordem. Produza as respostas sem duplicatas durante a geração, sem usar set. A única permutação de [] é a própria lista vazia, portanto sua saída é [[]].

Entrada `permutacoes([2, 1, 1])`
Saída `[[1, 1, 2], [1, 2, 1], [2, 1, 1]]`

Entrada `permutacoes([3, 3])`
Saída `[[3, 3]]`

Entrada `permutacoes([])`
Saída `[[]]`

24 Troco mínimo com desempate

DIFÍCIL

`troco(moedas, valor)`

Encontre uma forma de compor valor usando a menor quantidade possível de moedas, com uso ilimitado de cada denominação. Retorne [quantidade, escolhidas], com escolhidas em ordem crescente. Em empate, use a lista lexicograficamente menor.

Condições. As denominações são inteiros positivos distintos, em qualquer ordem; valor é inteiro não negativo. Se for impossível, retorne None. Para valor 0, retorne [0, []].

Entrada `troco([1, 3, 4], 6)`
Saída `[2, [3, 3]]`

Entrada `troco([1, 2, 3], 4)`
Saída `[2, [1, 3]]`

Entrada `troco([2, 4], 7)`
Saída `None`

25 Mochila 0/1

DIFÍCIL

`mochila(itens, capacidade)`

Cada item é representado por [peso, valor]. Retorne o maior valor total que pode ser transportado sem exceder a capacidade. Cada ocorrência da lista representa um item disponível uma única vez: não é permitido fracionar nem reutilizar.

Condições. Pesos são inteiros positivos; valores e capacidade são inteiros não negativos. É permitido escolher nenhum item, obtendo valor 0. Retorne apenas o valor ótimo.

```
Entrada  mochila([[2, 6], [3, 10], [4, 12]], 5)
Saída    16

Entrada  mochila([[2, 6]], 4)
Saída    6

Entrada  mochila([], 3)
Saída    0
```

26 Todas as soluções de N rainhas

DIFÍCIL

`rainhas(n)`

Posicione n rainhas em um tabuleiro n por n, sem ataques entre elas. Represente cada solução por uma lista de n colunas: o elemento na posição r indica a coluna da rainha da linha r. Colunas e linhas começam em zero.

Condições. n é inteiro não negativo. Retorne todas as soluções em ordem lexicográfica, mantendo soluções simétricas. Para n = 0, retorne [[]]. Não modifique um tabuleiro em memória.

```
Entrada  rainhas(4)
Saída    [[1, 3, 0, 2], [2, 0, 3, 1]]

Entrada  rainhas(2)
Saída    []

Entrada  rainhas(0)
Saída    [[]]
```

27 Contagem de caminhos com obstáculos

DIFÍCIL

`contar_caminhos(grade)`

Conte os caminhos do canto superior esquerdo ao canto inferior direito de uma grade. Só é permitido mover uma célula para a direita ou para baixo. Células com 0 são livres; células com 1 são bloqueadas e não podem ser visitadas.

Condições. A grade é [] ou uma matriz retangular não vazia de zeros e uns. Entrada vazia ou origem/destino bloqueado produz 0. Uma única célula livre tem um caminho.

Entrada `contar_caminhos([[0, 0, 0], [0, 1, 0], [0, 0, 0]])`
Saída `2`

Entrada `contar_caminhos([[1]])`
Saída `0`

Entrada `contar_caminhos([[0]])`
Saída `1`

28 Caminhos simples em um grafo dirigido

DIFÍCIL

`caminhos(grafo, origem, destino)`

O grafo é uma lista de registros [vértice, vizinhos]. Retorne todos os caminhos da origem ao destino sem repetir um vértice no mesmo caminho. Explore os vizinhos na ordem em que aparecem, em profundidade, para determinar a ordem da saída.

Condições. Vértices são inteiros únicos; vizinhos não se repetem, e todo vértice citado tem registro. Origem e destino existem. Ao atingir o destino, encerre aquele caminho. Ciclos não podem provocar recursão infinita.

Entrada `caminhos([[1, [2, 3]], [2, [3, 4]], [3, [1, 4]], [4, []]], 1, 4)`
Saída `[[1, 2, 3, 4], [1, 2, 4], [1, 3, 4]]`

Entrada `caminhos([[1, [2]], [2, []]], 2, 1)`
Saída `[]`

Entrada `caminhos([[1, [1]]], 1, 1)`
Saída `[[1]]`

29 Análise estrutural de uma árvore binária

DIFÍCIL

`analisar_arvore(arvore)`

Uma árvore é [] ou [valor, esquerda, direita]. Retorne [quantidade_de_nos, altura, eh_busca, eh_balanceada]. Para ser de busca, todo valor à esquerda deve ser menor que o nó e todo valor à direita, maior, em todos os níveis.

Condições. Valores são inteiros. A altura de [] é 0. A árvore é balanceada quando, em cada nó, as alturas das subárvores diferem no máximo em 1. [] é de busca e balanceada. Valores iguais violam a propriedade de busca.

Entrada `analisar_arvore([10, [5, [], [12, [], []]], [15, [], []])`
Saída `[4, 3, False, True]`

Entrada `analisar_arvore([3, [2, [1, [], []], []], []])`
Saída `[3, 3, True, False]`

Entrada `analisar_arvore([])`
Saída `[0, 0, True, True]`

30 Derivação simbólica com simplificação

DIFÍCIL

`derivar(expressao, variavel)`

Represente constantes por ['c', n], variáveis por ['v', nome], somas por ['+', a, b] e produtos por ['*', a, b]. Retorne a árvore da derivada em relação à variável indicada, usando as regras da soma e do produto: D(a*b) = D(a)*b + a*D(b).

Condições. n é inteiro e nomes são strings. Simplifique recursivamente: calcule operações entre constantes; elimine somas com 0 e produtos com 0 ou 1. Preserve a ordem dos operandos e não reúna termos semelhantes. Outras variáveis têm derivada 0.

Entrada `derivar(['*', ['v', 'x'], ['v', 'x']], 'x')`
Saída `['+', ['v', 'x'], ['v', 'x']]`

Entrada `derivar(['*', ['c', 3], ['v', 'x']], 'x')`
Saída `['c', 3]`

Entrada `derivar(['+', ['v', 'y'], ['c', 7]], 'x')`
Saída `['c', 0]`